Rockchip MIPI DSI 接口多供屏兼容开发指南

文件标识：RK-KF-YF-E26

发布版本：V1.1.0

日期：2025-05-16

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2025 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

产品版本

芯片名称	U-Boot 版本
RK3288	next-dev
RK3399	next-dev
RK3326/PX30	next-dev
RK356x	next-dev
RK3588	next-dev
RK3576	next-dev

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	黄国椿	2025-03-06	初始发布
V1.1.0	黄国椿	2025-05-16	修复获取 panel id 后 LPDT 通信异常

目录

Rockchip MIPI DSI 接口多供屏兼容开发指南

1. 概述
2. 核心功能说明
   - 2.1 多屏兼容机制
     - 2.1.1 动态面板检测
     - 2.1.2 设备树热修复
   - 2.2 设备树配置
     - 2.2.1 多面板节点定义
     - 2.2.2 面板 ID 配置
   - 2.3 驱动和设备树联动流程
     - 2.3.1 屏幕检测与切换流程
     - 2.3.2 完整补丁
3. 调试
   - 3.1 关键日志分析
   - 3.2 FDT 调试

1. 概述

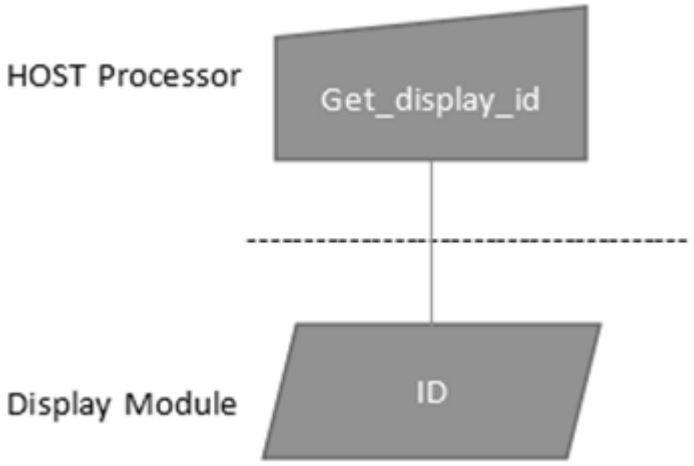
本指南旨在帮助开发人员和使用者在 Rockchip 含有 MIPI DSI 接口的 SoC 平台上配置和启用对多个多供 MIPI 显示屏的兼容支持。多供屏指的是项目中备用的不同供应商提供的显示屏，本指南将指导如何通过驱动层适配与设备树配置来实现同一个主板硬件环境搭配这些多供屏显示屏在同一个固件下能够动态检测切换并显示。

2. 核心功能说明

2.1 多屏兼容机制

2.1.1 动态面板检测

通过 MIPI DSI Lane0 双向传输的功能特性，读取对端屏幕的 ID 信息，自动匹配预定义的显示屏设备树配置描述。



2.1.2 设备树热修复

在 U-Boot 阶段的显示系统使能显示通路之前，先获取对端显示屏的 ID，根据匹配结果动态调整设备树节点状态（"status"），仅激活匹配的显示屏设备树配置（初始化序列、分辨率信息等）。

2.2 设备树配置

2.2.1 多面板节点定义

在 DTS 中对所有的多供屏的上电控制供电使能、RST、背光、MIPI DSI 工作模式、通道数、是否使能 DSC、DPHY 或 CPHY、屏初始化序列和对应屏分辨率等信息进行描述。

```
&dsi0 {
    status = "okay";

    dsi0_panel0: panel0@0 {
        status = "okay";
        compatible = "simple-panel-dsi";
        reg = <0>;

        dsi,flags = <(MIPI_DSI_MODE_LPM | MIPI_DSI_MODE_EOT_PACKET)>;
        dsi,format = <MIPI_DSI_FMT_RGB888>;
        dsi,lanes  = <4>;

        slice-width = <720>;
        slice-height = <65>;
        version-major = <1>;
        version-minor = <1>;
        compressed-data;

        id-reg = <0xdb>;
        id = [0x10];

        panel-init-sequence = [
            /* panel0 init sequence */
        ];

        panel-exit-sequence = [
            /* panel0 exit sequence */
        ];

        disp0_timings0: display-timings {
            native-mode = <&dsi_timing0>;
            dsi_timing0: timing0 {
                clock-frequency = <280000000>;
                hactive = <1440>;
                vactive = <3120>;
                hfront-porch = <16>;
                hsync-len = <8>;
                hback-porch = <8>;
                vfront-porch = <4>;
                vsync-len = <16>;
                vback-porch = <8>;
                hsync-active = <0>;
                vsync-active = <0>;
                de-active = <0>;
```

```
            pixelclk-active = <0>;
        };
    };

    ports {
        #address-cells = <1>;
        #size-cells = <0>;

        port@0 {
            reg = <0>;
            panel0_in_dsi0: endpoint {
                remote-endpoint = <&dsi0_out_panel>;
            };
        };
    };
};

dsi0_panel1: panel1@0 {
    status = "okay";
    compatible = "simple-panel-dsi";
    reg = <0>;

    dsi,flags = <(MIPI_DSI_MODE_VIDEO | MIPI_DSI_MODE_VIDEO_BURST |
        MIPI_DSI_MODE_LPM | MIPI_DSI_MODE_EOT_PACKET)>;
    dsi,format = <MIPI_DSI_FMT_RGB888>;
    dsi,lanes  = <2>;

    id-reg = <0xdb>;
    id = [0x72];

    panel-init-sequence = [
        /* panel1 init sequence */
    ];

    panel-exit-sequence = [
        /* panel1 exit sequence */
    ];

    disp0_timings1: display-timings {
        native-mode = <&dsi_timing1>;
        dsi_timing1: timing0 {
            clock-frequency = <110000000>;
            hactive = <1080>;
            vactive = <1920>;
            hfront-porch = <15>;
            hsync-len = <2>;
            hback-porch = <30>;
            vfront-porch = <15>;
            vsync-len = <2>;
            vback-porch = <15>;
            hsync-active = <0>;
            vsync-active = <0>;
            de-active = <0>;
```

```
            pixelclk-active = <1>;
        };
    };

    ports {
        #address-cells = <1>;
        #size-cells = <0>;

        port@0 {
            reg = <0>;
            panel1_in_dsi0: endpoint {
                remote-endpoint = <&dsi0_out_panel>;
            };
        };
    };
  };

  /* more panel */

  ports {
    #address-cells = <1>;
    #size-cells = <0>;

    port@1 {
        reg = <1>;
        dsi0_out_panel: endpoint {
            remote-endpoint = <&panel0_in_dsi0>;
        };
    };
  };

};
```

2.2.2 面板 ID 配置

先向屏厂确认获取各个多供屏 ID 的命令和预期 ID，并实际在屏端实践下是否能够按预期成功获取，然后将 ID 参数描述到 DTS 中。

属性	作用	示例值
id-reg	存储屏端 ID 寄存器地址	<0xdb> (寄存器 0xdb，不同屏地址一般不同)
id	预期 ID 值（十六进制数组）	[10] (长度也可能根据屏有差异)

2.3 驱动和设备树联动流程

2.3.1 屏幕检测与切换流程

```
开始
 |
 v
[驱动加载]
 |
 v
[遍历 DSI 控制器下所有 panel 节点，存储对应的信息]
 |
 v
[预初始化（pre_prepare）]
 |
 |-- [初始化 MIPI DSI]
 |
 v
[ID 匹配阶段]
 |
 |-- [发送 panel-read-id-sequence（若定义）]
 |-- [通过 DCS 读取 id-reg 的值]
 |-- [与设备树中 id 值比对]
 |-- [记录匹配的面板索引]
 |
 v
[动态设备树修复（fixup_dts）]
 |
 |-- [激活匹配面板的 remote-endpoint]
 |-- [禁用未匹配面板节点：fdt_setprop_string(..., "status", "disabled")]
 |
 v
[显示初始化]
 |
 |-- [使用匹配面板的时序和命令序列初始化]
 |
 v
结束
```

由于多屏兼容逻辑是在 U-Boot 阶段事先获取屏端 ID 并遍历设备树中 &dsi 节点下的所有 panel 节点，因此对开机时间有一定的开销(主要是屏端的上电时序延迟)，并且多屏兼容依赖 u-boot 开机 LOGO 功能。

2.3.2 完整补丁

```
From ace0a1101133a65dd535043437747916c3f331dc Mon Sep 17 00:00:00 2001
From: Zhibin Huang <zhibin.huang@rock-chips.com>
Date: Sat, 23 Mar 2024 21:39:32 +0800
Subject: [PATCH] video/drm: support multi mipi screen
```

```
Type: Function
Redmine ID: N/A
Associated modifications: N/A
Test: N/A

Signed-off-by: Zhibin Huang <zhibin.huang@rock-chips.com>
Change-Id: I4abe3e8bf6a4b226bc400054fa1c0ba523aafd93
---

diff --git a/drivers/video/drm/dw_mipi_dsi.c b/drivers/video/drm/dw_mipi_dsi.c
index 2ebe826..6e13b24 100644
--- a/drivers/video/drm/dw_mipi_dsi.c
+++ b/drivers/video/drm/dw_mipi_dsi.c
@@ -18,6 +18,7 @@
 #include <syscon.h>
 #include <asm/arch-rockchip/clock.h>
 #include <linux/iopoll.h>
+#include <fdt_support.h>

 #include "rockchip_display.h"
 #include "rockchip_crtc.h"
@@ -1259,6 +1260,46 @@
       dw_mipi_dsi_pre_enable(dsi->slave);
 }

+static void dw_mipi_dsi_connector_pre_prepare(struct rockchip_connector *conn,
+                      struct display_state *state)
+{
+   struct connector_state *conn_state = &state->conn_state;
+   struct dw_mipi_dsi *dsi = dev_get_priv(conn->dev);
+   unsigned long lane_rate = 1000000000UL;
+
+   memcpy(&dsi->mode, &conn_state->mode, sizeof(struct drm_display_mode));
+   if (dsi->slave)
+      memcpy(&dsi->slave->mode, &dsi->mode,
+             sizeof(struct drm_display_mode));
+
+   if (dsi->dphy.phy)
+      dw_mipi_dsi_set_hs_clk(dsi, lane_rate);
+   else
+      dw_mipi_dsi_set_pll(dsi, lane_rate);
+
+   if (dsi->slave && dsi->slave->dphy.phy)
+      dw_mipi_dsi_set_hs_clk(dsi->slave, lane_rate);
+
+   printf("final DSI-Link bandwidth: %u Mbps x %d\n",
+          dsi->lane_mbps, dsi->slave ? dsi->lanes * 2 : dsi->lanes);
+
+   dw_mipi_dsi_init(dsi);
+   dw_mipi_dsi_dpi_config(dsi, &dsi->mode);
+   dw_mipi_dsi_packet_handler_config(dsi);
+   dw_mipi_dsi_video_mode_config(dsi);
```

```
+	dw_mipi_dsi_command_mode_config(dsi);
+	dsi_update_bits(dsi, DSI_MODE_CFG, CMD_VIDEO_MODE, COMMAND_MODE);
+
+	dw_mipi_dsi_dphy_timing_config(dsi);
+	dw_mipi_dsi_dphy_interface_config(dsi);
+
+	dw_mipi_dsi_clear_err(dsi);
+
+	mipi_dphy_init(dsi);
+	mipi_dphy_power_on(dsi);
+	dsi_write(dsi, DSI_PWR_UP, POWERUP);
+}
+
 static int dw_mipi_dsi_connector_prepare(struct rockchip_connector *conn,
 					 struct display_state *state)
 {
@@ -1320,12 +1361,50 @@
 	return 0;
 }

+static void dw_mipi_dsi_connector_fixup_dts(struct rockchip_connector *conn,
+					    struct display_state *state)
+{
+	struct panel_state *panel_state = &state->panel_state;
+	struct rockchip_panel *panel = panel_state->panel;
+	ofnode *rockchip_panel_node = &panel->dev->node;
+	const struct device_node *conn_node;
+	char conn_endpoint_path[128], panel_endpoint_path[128];
+	const char *conn_path, *panel_path;
+	void *blob = (void*)gd->fdt_blob;
+	ofnode node;
+	u32 phandle = 0;
+	int offset;
+
+	conn_node = ofnode_to_np(conn->dev->node);
+	conn_path = conn_node->full_name;
+	strcpy(conn_endpoint_path, conn_path);
+	strcat(conn_endpoint_path, "/ports/port@1/endpoint");
+
+	panel_path = rockchip_panel_node->np->full_name;
+	strcpy(panel_endpoint_path, panel_path);
+	strcat(panel_endpoint_path, "/ports/port@0/endpoint");
+	phandle = fdt_get_phandle(blob, fdt_path_offset(blob, panel_endpoint_path));
+	do_fixup_by_path_u32(blob, (const char*)conn_endpoint_path,
+			     "remote-endpoint", phandle, 1);
+
+	ofnode_for_each_subnode(node, conn->dev->node) {
+		if (strstr(node.np->name, "panel") &&
+		    ofnode_valid(node) && ofnode_is_available(node) &&
+		    strcmp(node.np->full_name, panel_path)) {
+			offset = fdt_path_offset(blob, node.np->full_name);
+			fdt_setprop_string(blob, offset, "status", "disa");
+		}
```

```
+   }
+}
+
 static const struct rockchip_connector_funcs dw_mipi_dsi_connector_funcs = {
    .init = dw_mipi_dsi_connector_init,
    .prepare = dw_mipi_dsi_connector_prepare,
    .unprepare = dw_mipi_dsi_connector_unprepare,
    .enable = dw_mipi_dsi_connector_enable,
    .disable = dw_mipi_dsi_connector_disable,
+   .pre_prepare = dw_mipi_dsi_connector_pre_prepare,
+   .fixup_dts = dw_mipi_dsi_connector_fixup_dts,
 };

 static int dw_mipi_dsi_probe(struct udevice *dev)
diff --git a/drivers/video/drm/dw_mipi_dsi2.c b/drivers/video/drm/dw_mipi_dsi2.c
index ecd086d7d62..b62fafae370 100644
--- a/drivers/video/drm/dw_mipi_dsi2.c
+++ b/drivers/video/drm/dw_mipi_dsi2.c
@@ -555,6 +555,8 @@ static void dw_mipi_dsi2_ipi_color_coding_cfg(struct dw_mipi_dsi2 *dsi2)

    if (dsi2->dsc_enable)
        grf_field_write(dsi2, IPI_FORMAT, IPI_FORMAT_DSC);
+   else
+       grf_field_write(dsi2, IPI_FORMAT, IPI_FORMAT_RGB);
 }

 static void dw_mipi_dsi2_ipi_set(struct dw_mipi_dsi2 *dsi2)
@@ -719,11 +721,13 @@ static void dw_mipi_dsi2_disable(struct dw_mipi_dsi2 *dsi2)
        dw_mipi_dsi2_disable(dsi2->slave);
 }

+static void dw_mipi_dsi2_host_softrst(struct dw_mipi_dsi2 *dsi2);
 static void dw_mipi_dsi2_post_disable(struct dw_mipi_dsi2 *dsi2)
 {
    if (!dsi2->prepared)
        return;

+   dw_mipi_dsi2_host_softrst(dsi2);
    dsi_write(dsi2, DSI2_PWR_UP, RESET);

    if (dsi2->dcphy.phy)
@@ -853,6 +857,9 @@ static int dw_mipi_dsi2_connector_init(struct rockchip_connector *conn, struct d
    if (!(dsi2->mode_flags & MIPI_DSI_MODE_VIDEO)) {
        conn_state->output_flags |= ROCKCHIP_OUTPUT_MIPI_DS_MODE;
        conn_state->hold_mode = dsi2->disable_hold_mode ? false : true;
+   } else {
+       conn_state->output_flags &= ~ROCKCHIP_OUTPUT_MIPI_DS_MODE;
+       conn_state->hold_mode = false;
    }

    if (dsi2->dual_channel) {
@@ -912,6 +919,8 @@ static int dw_mipi_dsi2_connector_init(struct rockchip_connector *conn, struct d
        cstate->dsc_sink_cap.target_bits_per_pixel_x16 = 8 << 4;
```

```
      cstate->dsc_sink_cap.native_420 = 0;
      memcpy(&cstate->pps, dsi2->pps, sizeof(struct drm_dsc_picture_parameter_set));
+   } else {
+      cstate->dsc_enable = 0;
    }

    return 0;
@@ -1185,6 +1194,31 @@ static void dw_mipi_dsi2_get_mipi_pixel_clk(struct dw_mipi_dsi2 *dsi2,
      dsi2->slave->mipi_pixel_rate = dsi2->mipi_pixel_rate;
}

+static void dw_mipi_dsi2_connector_pre_prepare(struct rockchip_connector *conn,
+                       struct display_state *state)
+{
+   struct dw_mipi_dsi2 *dsi2 = dev_get_priv(conn->dev);
+   struct connector_state *conn_state = &state->conn_state;
+   unsigned long lane_rate = 1000000000UL;
+
+   memcpy(&dsi2->mode, &conn_state->mode, sizeof(struct drm_display_mode));
+   if (dsi2->slave)
+      memcpy(&dsi2->slave->mode, &dsi2->mode,
+             sizeof(struct drm_display_mode));
+
+   if (dsi2->dcphy.phy)
+      dw_mipi_dsi2_set_hs_clk(dsi2, lane_rate);
+
+   if (dsi2->slave && dsi2->slave->dcphy.phy)
+      dw_mipi_dsi2_set_hs_clk(dsi2->slave, lane_rate);
+
+   printf("final DSI-Link bandwidth: %u %s x %d\n",
+          dsi2->lane_hs_rate, dsi2->c_option ? "Ksps" : "Kbps",
+          dsi2->slave ? dsi2->lanes * 2 : dsi2->lanes);
+
+   dw_mipi_dsi2_pre_enable(dsi2);
+}
+
static int dw_mipi_dsi2_connector_prepare(struct rockchip_connector *conn,
                     struct display_state *state)
{
@@ -1281,6 +1315,42 @@ static int dw_mipi_dsi2_connector_mode_valid(struct rockchip_connector *conn,
    return MODE_OK;
 }

+static void dw_mipi_dsi2_connector_fixup_dts(struct rockchip_connector *conn,
+                       struct display_state *state)
+{
+   struct panel_state *panel_state = &state->panel_state;
+   struct rockchip_panel *panel = panel_state->panel;
+   ofnode *rockchip_panel_node = &panel->dev->node;
+   const struct device_node *conn_node;
+   char conn_endpoint_path[128], panel_endpoint_path[128];
+   const char *conn_path, *panel_path;
+   void *blob = (void*)gd->fdt_blob;
```

```
+   ofnode node;
+   u32 phandle = 0;
+   int offset;
+
+   conn_node = ofnode_to_np(conn->dev->node);
+   conn_path = conn_node->full_name;
+   strcpy(conn_endpoint_path, conn_path);
+   strcat(conn_endpoint_path, "/ports/port@1/endpoint");
+
+   panel_path = rockchip_panel_node->np->full_name;
+   strcpy(panel_endpoint_path, panel_path);
+   strcat(panel_endpoint_path, "/ports/port@0/endpoint");
+   phandle = fdt_get_phandle(blob, fdt_path_offset(blob, panel_endpoint_path));
+   do_fixup_by_path_u32(blob, (const char*)conn_endpoint_path,
+               "remote-endpoint", phandle, 1);
+
+   ofnode_for_each_subnode(node, conn->dev->node) {
+       if (strstr(node.np->name, "panel") &&
+           ofnode_valid(node) && ofnode_is_available(node) &&
+           strcmp(node.np->full_name, panel_path)) {
+           offset = fdt_path_offset(blob, node.np->full_name);
+           fdt_setprop_string(blob, offset, "status", "disa");
+       }
+   }
+}
+
static const struct rockchip_connector_funcs dw_mipi_dsi2_connector_funcs = {
    .pre_init = dw_mipi_dsi2_connector_pre_init,
    .init = dw_mipi_dsi2_connector_init,
@@ -1289,6 +1359,8 @@ static const struct rockchip_connector_funcs dw_mipi_dsi2_connector_funcs = {
    .enable = dw_mipi_dsi2_connector_enable,
    .disable = dw_mipi_dsi2_connector_disable,
    .mode_valid = dw_mipi_dsi2_connector_mode_valid,
+   .pre_prepare = dw_mipi_dsi2_connector_pre_prepare,
+   .fixup_dts = dw_mipi_dsi2_connector_fixup_dts,
 };

static int dw_mipi_dsi2_probe(struct udevice *dev)
diff --git a/drivers/video/drm/rockchip_connector.h b/drivers/video/drm/rockchip_connector.h
index 8c2ff75..e098b7f 100644
--- a/drivers/video/drm/rockchip_connector.h
+++ b/drivers/video/drm/rockchip_connector.h
@@ -183,6 +183,12 @@
     */
    int (*get_edid)(struct rockchip_connector *connector, struct display_state *state);
    /*
+    * for reading panel id,configure mipi host and phy simply
+    * so that reading and writing command can send succeessfully.
+    */
+   void (*pre_prepare)(struct rockchip_connector *connector, struct display_state *state);
+   void (*fixup_dts)(struct rockchip_connector *connector, struct display_state *state);
+   /*
     * call before crtc enable.
```

```
     */
    int (*prepare)(struct rockchip_connector *connector, struct display_state *state);
diff --git a/drivers/video/drm/rockchip_display.c b/drivers/video/drm/rockchip_display.c
index ae99b4d..1042f41 100644
--- a/drivers/video/drm/rockchip_display.c
+++ b/drivers/video/drm/rockchip_display.c
@@ -762,6 +762,15 @@
    if (!ret && !state->force_output)
        goto deinit;

+   if (conn->panel) {
+       rockchip_panel_pre_prepare(conn->panel);
+       if (state->enabled_at_spl == false) {
+           ret = rockchip_connector_init(state);
+           if (ret)
+               goto deinit;
+       }
+   }
+
    ret = 0;
    if (state->enabled_at_spl == true) {
 #ifdef CONFIG_SPL_BUILD
@@ -1736,12 +1745,17 @@
    int ret = 0;

    *panel = NULL;
-   panel_ofnode = dev_read_subnode(dev, "panel");
-   if (ofnode_valid(panel_ofnode) && ofnode_is_available(panel_ofnode)) {
-       ret = uclass_get_device_by_ofnode(UCLASS_PANEL, panel_ofnode,
-                         &panel_dev);
-       if (!ret)
-           goto found;
+   ofnode_for_each_subnode(panel_ofnode, dev->node) {
+       if (!strstr(panel_ofnode.np->name, "panel"))
+           continue;
+       if (ofnode_valid(panel_ofnode) &&
+           ofnode_is_available(panel_ofnode)) {
+           ret = uclass_get_device_by_ofnode(UCLASS_PANEL,
+                             panel_ofnode,
+                             &panel_dev);
+           if (!ret)
+               goto found;
+       }
    }

    ep_node = rockchip_of_graph_get_remote_node(dev->node, PORT_DIR_OUT, 0);
diff --git a/drivers/video/drm/rockchip_panel.c b/drivers/video/drm/rockchip_panel.c
index 9ddb4d8..f4b91c5 100644
--- a/drivers/video/drm/rockchip_panel.c
+++ b/drivers/video/drm/rockchip_panel.c
@@ -15,16 +15,20 @@
 #include <spi.h>
 #include <asm/gpio.h>
```

```
 #include <dm/device.h>
+#include <dm/device-internal.h>
 #include <dm/read.h>
 #include <dm/uclass.h>
 #include <dm/uclass-id.h>
 #include <linux/media-bus-format.h>
 #include <power/regulator.h>
+#include <dm/of_access.h>
+#include <fdt_support.h>

 #include "rockchip_display.h"
 #include "rockchip_crtc.h"
 #include "rockchip_connector.h"
 #include "rockchip_panel.h"
+#include "rockchip_phy.h"

 struct rockchip_cmd_header {
    u8 data_type;
@@ -42,6 +46,11 @@
    int cmd_cnt;
 };

+struct dsi_panel_id {
+   u8 *buf;
+   int len;
+};
+
 struct rockchip_panel_plat {
    bool power_invert;
    u32 bus_format;
@@ -58,6 +67,10 @@

    struct rockchip_panel_cmds *on_cmds;
    struct rockchip_panel_cmds *off_cmds;
+   struct rockchip_panel_cmds *read_id_cmds;
+
+   unsigned int id_reg;
+   struct dsi_panel_id *id;
 };

 struct rockchip_panel_priv {
@@ -411,7 +424,205 @@
    priv->enabled = false;
 }

+static int panel_simple_dsi_match_id(struct rockchip_panel *panel)
+{
+   struct rockchip_panel_plat *plat = dev_get_platdata(panel->dev);
+   struct rockchip_panel_priv *priv = dev_get_priv(panel->dev);
+   struct mipi_dsi_device *dsi = dev_get_parent_platdata(panel->dev);
+   int cmd_len = plat->id->len;
+   int read_count = 1, match_count;
+   int ret = -EINVAL;
```

```
+	u8 *panel_id;
+
+	panel_id = devm_kzalloc(panel->dev, sizeof(char) * cmd_len, GFP_KERNEL);
+	if (!panel_id)
+		return -ENOMEM;
+
+	if (priv->power_supply)
+		regulator_set_enable(priv->power_supply, !plat->power_invert);
+
+	if (dm_gpio_is_valid(&priv->enable_gpio))
+		dm_gpio_set_value(&priv->enable_gpio, 1);
+
+	if (plat->delay.prepare)
+		mdelay(plat->delay.prepare);
+
+	if (dm_gpio_is_valid(&priv->reset_gpio))
+		dm_gpio_set_value(&priv->reset_gpio, 1);
+
+	if (plat->delay.reset)
+		mdelay(plat->delay.reset);
+
+	if (dm_gpio_is_valid(&priv->reset_gpio))
+		dm_gpio_set_value(&priv->reset_gpio, 0);
+
+	if (plat->delay.init)
+		mdelay(plat->delay.init);
+
+	if (plat->read_id_cmds) {
+		rockchip_panel_send_dsi_cmds(dsi, plat->read_id_cmds);
+		free(plat->read_id_cmds);
+		plat->read_id_cmds = NULL;
+	}
+
+	if (cmd_len > 1)
+		mipi_dsi_set_maximum_return_packet_size(dsi, cmd_len);
+
+	for (int i = 0; i < read_count; i++) {
+
+		printf("panel id, reg: 0x%02x, val: ", plat->id_reg);
+		for (int j = 0; j < cmd_len; j++)
+			printf("%02x", plat->id->buf[j]);
+		printf("\n");
+		ret = mipi_dsi_dcs_read(dsi, plat->id_reg, panel_id, cmd_len);
+		if (ret > 0) {
+			printf("read panel id, reg: 0x%02x, val: ", plat->id_reg);
+			match_count = 0;
+			for (int j = 0; j < cmd_len; j++) {
+				printf("%02x", panel_id[j]);
+				if (panel_id[j] == plat->id->buf[j])
+					match_count++;
+			}
+			printf("\n");
+			if (match_count == cmd_len) {
```

```
+            ret = 0;
+            break;
+        }
+        ret = -EINVAL;
+    } else {
+        printf("read panel timeout or no response\n");
+        ret = -ETIMEDOUT;
+    }
+ }
+
+ if (dm_gpio_is_valid(&priv->reset_gpio))
+    dm_gpio_set_value(&priv->reset_gpio, 1);
+
+ if (dm_gpio_is_valid(&priv->enable_gpio))
+    dm_gpio_set_value(&priv->enable_gpio, 0);
+
+ if (priv->power_supply)
+    regulator_set_enable(priv->power_supply, plat->power_invert);
+
+ if (plat->delay.unprepare)
+    mdelay(plat->delay.unprepare);
+
+ return ret;
+}
+
+static int panel_simple_dsi_match(struct display_state *state, int panel_num)
+{
+ struct connector_state *conn_state = &state->conn_state;
+ struct panel_state *panel_state = &state->panel_state;
+ struct rockchip_connector *conn = conn_state->connector;
+ const struct rockchip_connector_funcs *conn_funcs = conn->funcs;
+ struct rockchip_panel *panel;
+ ofnode panel_ofnode;
+ struct udevice *panel_dev;
+ int ret;
+
+ if (conn_funcs->pre_init) {
+    ret = conn_funcs->pre_init(conn, state);
+    if (ret) {
+        printf("failed to pre_init connector\n");
+        goto deinit;
+    }
+ }
+
+ if (conn_funcs->init) {
+    ret = conn_funcs->init(conn, state);
+    if (ret) {
+        printf("failed to init connector\n");
+        goto deinit;
+    }
+ }
+
+ if (conn->phy)
```

```
+	rockchip_phy_init(conn->phy);
+
+	if (conn_funcs->pre_prepare)
+		conn_funcs->pre_prepare(conn, state);
+
+	ofnode_for_each_subnode(panel_ofnode, conn->dev->node) {
+		if (!strstr(panel_ofnode.np->name, "panel"))
+			continue;
+		if (ofnode_valid(panel_ofnode) &&
+			ofnode_is_available(panel_ofnode)) {
+			ret = uclass_get_device_by_ofnode(UCLASS_PANEL,
+					panel_ofnode,
+					&panel_dev);
+			if (ret) {
+				printf("this sub node is not a panel node\n");
+				continue;
+			}
+		} else {
+			continue;
+		}
+
+		panel = (struct rockchip_panel *)dev_get_driver_data(panel_dev);
+		if (!panel) {
+			printf("can not found panel\n");
+			ret = -ENODEV;
+			break;
+		}
+
+		panel_state->panel = panel;
+		conn->panel = panel;
+		panel->state = state;
+
+		printf("try to match %s\n", panel_ofnode.np->full_name);
+		ret = panel_simple_dsi_match_id(panel);
+		if (!ret)
+			break;
+
+		if (panel_num > 1) {
+			device_remove(panel_dev, DM_REMOVE_NORMAL);
+			panel_num--;
+		}
+	}
+
+	if (conn_funcs->disable)
+		conn_funcs->disable(conn, state);
+
+	if (conn_funcs->unprepare)
+		conn_funcs->unprepare(conn, state);
+
+	if (conn_funcs->fixup_dts)
+		conn_funcs->fixup_dts(conn, state);
+
+deinit:
```

```
+	if (conn_funcs->deinit)
+		conn_funcs->deinit(conn, state);
+
+	return ret;
+}
+
+static void panel_simple_pre_prepare(struct rockchip_panel *panel)
+{
+	struct display_state *state = panel->state;
+	struct connector_state *conn_state = &state->conn_state;
+	struct rockchip_connector *conn = conn_state->connector;
+	char *conn_name = (char *)conn->dev->node.np->full_name;
+	ofnode panel_ofnode;
+	int panel_num = 0;
+
+	conn_state->bus_format = panel->bus_format;
+
+	ofnode_for_each_subnode(panel_ofnode, conn->dev->node) {
+		if (strstr(panel_ofnode.np->name, "panel") &&
+		    ofnode_valid(panel_ofnode) &&
+		    ofnode_is_available(panel_ofnode))
+			panel_num++;
+	}
+
+	if (panel_num > 1 && strstr(conn_name, "dsi")) {
+		if (panel_simple_dsi_match(state, panel_num))
+			printf("fail to match valid panel\n");
+		else
+			printf("success to match valid panel\n");
+	}
+}
+
 static const struct rockchip_panel_funcs rockchip_panel_funcs = {
+	.pre_prepare = panel_simple_pre_prepare,
 	.prepare = panel_simple_prepare,
 	.unprepare = panel_simple_unprepare,
 	.enable = panel_simple_enable,
@@ -436,6 +647,17 @@

 	plat->bus_format = dev_read_u32_default(dev, "bus-format",
 						MEDIA_BUS_FMT_RBG888_1X24);
+	plat->id_reg = dev_read_u32_default(dev, "id-reg", 0);
+	data = dev_read_prop(dev, "id", &len);
+
+	if (data) {
+		plat->id = devm_kzalloc(dev, sizeof(*plat->id), GFP_KERNEL);
+		if (!plat->id)
+			return -ENOMEM;
+		plat->id->buf = devm_kzalloc(dev, sizeof(char) * len, GFP_KERNEL);
+		if (!plat->id->buf)
+			return -ENOMEM;
+
+		memcpy(plat->id->buf, data, len);
```

```
+       plat->id->len = len;
+   }
    plat->bpc = dev_read_u32_default(dev, "bpc", 8);

    data = dev_read_prop(dev, "panel-init-sequence", &len);
@@ -451,6 +673,19 @@
        }
    }

+   data = dev_read_prop(dev, "panel-read-id-sequence", &len);
+   if (data) {
+       plat->read_id_cmds = calloc(1, sizeof(*plat->read_id_cmds));
+       if (!plat->read_id_cmds)
+           return -ENOMEM;
+
+       ret = rockchip_panel_parse_cmds(data, len, plat->read_id_cmds);
+       if (ret) {
+           printf("failed to parse panel read id sequence\n");
+           goto free_read_id_cmds;
+       }
+   }
+
    data = dev_read_prop(dev, "panel-exit-sequence", &len);
    if (data) {
        plat->off_cmds = calloc(1, sizeof(*plat->off_cmds));
@@ -470,6 +705,8 @@

 free_cmds:
    free(plat->off_cmds);
+free_read_id_cmds:
+   free(plat->read_id_cmds);
 free_on_cmds:
    free(plat->on_cmds);
    return ret;
@@ -574,6 +811,19 @@
    return 0;
 }

+static int panel_simple_remove(struct udevice *dev) {
+   struct rockchip_panel_priv *priv = dev_get_priv(dev);
+
+   printf("remove %s\n", dev->node.np->full_name);
+
+   if (dm_gpio_is_valid(&priv->enable_gpio))
+       dm_gpio_free(dev, &priv->enable_gpio);
+   if (dm_gpio_is_valid(&priv->reset_gpio))
+       dm_gpio_free(dev, &priv->reset_gpio);
+
+   return 0;
+}
+
 static const struct udevice_id rockchip_panel_ids[] = {
    { .compatible = "simple-panel", },
```

```
   { .compatible = "simple-panel-dsi", },
@@ -587,6 +837,7 @@
   .of_match = rockchip_panel_ids,
   .ofdata_to_platdata = rockchip_panel_ofdata_to_platdata,
   .probe = rockchip_panel_probe,
+   .remove = panel_simple_remove,
   .priv_auto_alloc_size = sizeof(struct rockchip_panel_priv),
   .platdata_auto_alloc_size = sizeof(struct rockchip_panel_plat),
 };
diff --git a/drivers/video/drm/rockchip_panel.h b/drivers/video/drm/rockchip_panel.h
index 5e49fe3..b503e84 100644
--- a/drivers/video/drm/rockchip_panel.h
+++ b/drivers/video/drm/rockchip_panel.h
@@ -12,10 +12,12 @@
 struct rockchip_conn;

 struct rockchip_panel_funcs {
+   void (*pre_prepare)(struct rockchip_panel *panel);
   void (*prepare)(struct rockchip_panel *panel);
   void (*unprepare)(struct rockchip_panel *panel);
   void (*enable)(struct rockchip_panel *panel);
   void (*disable)(struct rockchip_panel *panel);
+   void (*init)(struct rockchip_panel *panel);
   int (*get_mode)(struct rockchip_panel *panel,
         struct drm_display_mode *mode);
 };
@@ -48,6 +50,15 @@
       state->conn_state.bpc = panel->bpc;
 }

+static inline void rockchip_panel_pre_prepare(struct rockchip_panel *panel)
+{
+   if (!panel)
+      return;
+
+   if (panel->funcs && panel->funcs->pre_prepare)
+      panel->funcs->pre_prepare(panel);
+}
+
 static inline void rockchip_panel_prepare(struct rockchip_panel *panel)
 {
   if (!panel)
```

3. 调试

3.1 关键日志分析

根据如下去排查屏端获取 ID 失败的原因：

```
# ID 匹配成功日志
read panel id, reg: 0xDA, val: 09     # 读取到的 ID 值
success to match valid panel         # 驱动确认匹配

# 异常场景
fail to read panel id                 # 通信失败
ID mismatch: expect 08 got 09         # ID 不匹配
```

3.2 FDT 调试

```
adb pull /sys/firmware/fdt ./fdt

在 kernel 根目录下执行 DTB 反编译
./scripts/dtc/dtc  -I dtb -O dts -o fdt.dts ./fdt
```

查看 fdt.dts 验证节点状态是否符合预期，比如：

```
panel0@0 { status = "okay"; ... };
panel1@0 { status = "disabled"; ... };
```